

Flash Sale — Gestion de Stock

Réservation courte + Anti-survente (Redis NoSQL)

Module : Bases de Données NoSQL
SGBD imposé : Redis (Base clé-valeur)

Étudiant (Chef de projet) : Khalid Morjan
Co-équipier : Anas Lahraoui
Filière : Systèmes d'Information & Intelligence Artificielle
Année universitaire : 2025–2026

Livrables : service (`app.py`) + couche Redis (`redis_client.py`) + config (`config.py`)
scripts de simulation (`simulate_clients.py`, `scenario_concurrency.py`) + logs
(`results_logs.txt`)
installation (`requirements.txt`) + Gantt + annexes.

Date : 7 février 2026

Table des matières

1	Résumé exécutif	3
2	Contexte, objectifs et périmètre	3
2.1	Contexte	3
2.2	Objectifs	3
2.3	Contraintes	3
3	Méthodologie Agile / Scrum	3
3.1	Organisation Scrum (4 semaines)	3
3.2	Rôles	4
3.3	Backlog (extraits)	4
3.4	Plan des sprints (résumé)	6
4	Architecture de la solution	8
4.1	Vue en couches	8
4.2	Schéma global	8
4.3	Choix techniques Redis	9
5	Modèle de données Redis (clés & structures)	9
5.1	Principe de gestion du stock réservé	9
5.2	Liste des clés	9
5.3	Justification (structures)	10
6	Implémentation des cas d'usage	10
6.1	Initialisation du stock	10
6.2	Réservation atomique (Lua)	10
6.3	Annulation (cancel)	10
6.4	Expiration et libération automatique (sweeper)	10
6.5	Paiement confirmé (idempotence)	11
7	Gestion des cas limites	11
8	Installation et exécution (Windows + Docker)	11
8.1	Prérequis	11
8.2	Démarrage Redis (Docker)	11
8.3	Variables d'environnement (PowerShell, ACL Redis)	11
8.4	Commande de santé	11
9	Tests, simulation de charge et preuves	12
9.1	Simulation multi-clients (simulate_clients.py)	12
9.2	Comparatif SAFE vs UNSAFE (scenario_concurrency.py)	12
9.3	Interprétation des résultats	12
9.4	Logs et preuves	12
10	Planification (Gantt)	12
11	Conclusion	13

A Annexes	13
A.1 Arborescence du projet	13
A.2 Commandes de démo	14

1 Résumé exécutif

Ce projet répond au besoin d'une marque effectuant des ventes flash (durée minutes/heures) où le risque majeur est la **survente** due aux accès concurrents (achats simultanés) et aux paniers non finalisés. La solution proposée repose sur **Redis** (base clé-valeur NoSQL) et met en œuvre :

- une **réservation temporaire** avec **TTL** (expiration automatique) ;
- une **atomicité** (script Lua) pour empêcher les conditions de course ;
- des mécanismes de **libération du stock** après expiration (sweeper) ;
- des **métriques** et un **audit** pour prouver la cohérence, mesurer l'abandon et tracer les opérations ;
- l'**idempotence** contre le double paiement.

2 Contexte, objectifs et périmètre

2.1 Contexte

Pendant une vente flash, de nombreux clients tentent d'acheter simultanément. Sans mécanismes robustes, deux clients peuvent réserver (ou payer) le même stock, provoquant une **survente**.

2.2 Objectifs

- Empêcher la survente sous forte concurrence.
- Réserver temporairement un stock au passage en caisse (panier) avec expiration.
- Confirmer à l'achat (paiement) et libérer automatiquement en cas d'abandon.
- Produire des indicateurs : taux d'abandon, stock réservé vs vendu, incidents.

2.3 Contraintes

- **Redis obligatoire** (structures + TTL).
- Prévention des incohérences : atomicité et gestion des cas limites.
- Simulation de charge multi-clients concurrents.
- Travail en mode **Agile/Scrum** (sprints, backlog, livrables).

3 Méthodologie Agile / Scrum

3.1 Organisation Scrum (4 semaines)

Le projet a été organisé en sprints (1 semaine par sprint) afin d'obtenir une version démontrable .

3.2 Rôles

- **Product Owner / Chef de projet** : Khalid Morjan (priorisation, backlog, soutenance).
- **Développeur** : Anas Lahraoui (implémentations ciblées, tests, docs).

3.3 Backlog (extraits)

Epic	User Story	Priorité	Statut
E1	En tant que système, je dois modéliser le stock et les réservations pour éviter la survente.	Haute	Done
E2	En tant que client, je réserve un stock temporairement avec expiration automatique.	Haute	Done
E3	En tant que client, je confirme le paiement de manière idempotente.	Haute	Done
E4	En tant que chef de projet, je dois simuler une charge concurrente et prouver l'anti-survente.	Haute	Done
E5	En tant qu'auditeur, je veux des métriques et logs exploitables dans le rapport.	Moyenne	Done

Work			
Sprints			
☐	▼ <u>FSR-1 Epic 1 – Conception & règles métier</u>		DONE
☐	FSR-11 Justification du choix Redis	DONE	🔗
☐	FSR-16 Analyse du problème métier (flash sale & s...	DONE	🔗
☐	FSR-14 Conception du workflow global	🔗	DONE
☐	FSR-48 Définition du modèle de données Redis pour...	DONE	🔗
☐	FSR-15 Identification des risques et cas limites	DONE	🔗
☐	FSR-12 Définition des règles métier	DONE	🔗
☐	▼ <u>FSR-23 Epic 2 – Réserveation et Expiration du Stock</u>		DONE
☐	✓ FSR-29 Documentation du Sprint 2 et validation d...	🔗	DONE
☐	✓ FSR-24 Conception du modèle de clés Redis pour...	🔗	DONE
☐	✓ FSR-25 Définition des règles d'expiration des rése...	🔗	DONE
☐	✓ FSR-28 Simulation de plusieurs clients réservant s...	🔗	DONE
☐	✓ FSR-27 Gestion de l'expiration et libération autom...	🔗	DONE
☐	✓ FSR-26 Implémentation de la réservation tempora...	🔗	DONE
☐	▼ <u>FSR-49 Epic 3 – Confirmation paiement & Indicateurs</u>		DONE
☐	FSR-58 Confirmation paiement Idempotente	DONE	🔗
☐	FSR-51 Indicateurs de performance (KPI)	DONE	🔗
☐	FSR-52 Audit des opérations	DONE	🔗
☐	FSR-53 Test du double paiement	DONE	🔗
☐	▼ <u>FSR-54 Epic 4 – Simulation de charge & preuve anti-survente</u>		DONE
☐	FSR-63 Simulation multi-clients	DONE	🔗
☐	FSR-64 Comparaison SAFE vs UNSAFE	DONE	🔗
☐	FSR-65 Analyse des résultats	DONE	🔗
☐	FSR-66 Rapport & preuves	DONE	🔗
☐	▼ <u>FSR-67 Epic 5 – Métriques, logs et exploitation dans le rapport</u>		
☐	✓ FSR-68 Compteurs système (Metrics)	DONE	🔗
☐	✓ FSR-69 Journal d'audit	DONE	🔗
☐	✓ FSR-70 Exploitation des logs dans le rapport	DONE	🔗
☐	✓ FSR-71 Détection d'anomalies	DONE	🔗

FIGURE 1 – Backlog Jira du projet montrant les Epics (E1–E5), les User Stories associées et leurs statuts

3.4 Plan des sprints (résumé)

- **Sprint 1 (Jour 1)** : Analyse métier, modèle de clés Redis, setup Docker/ACL, premier service CLI.

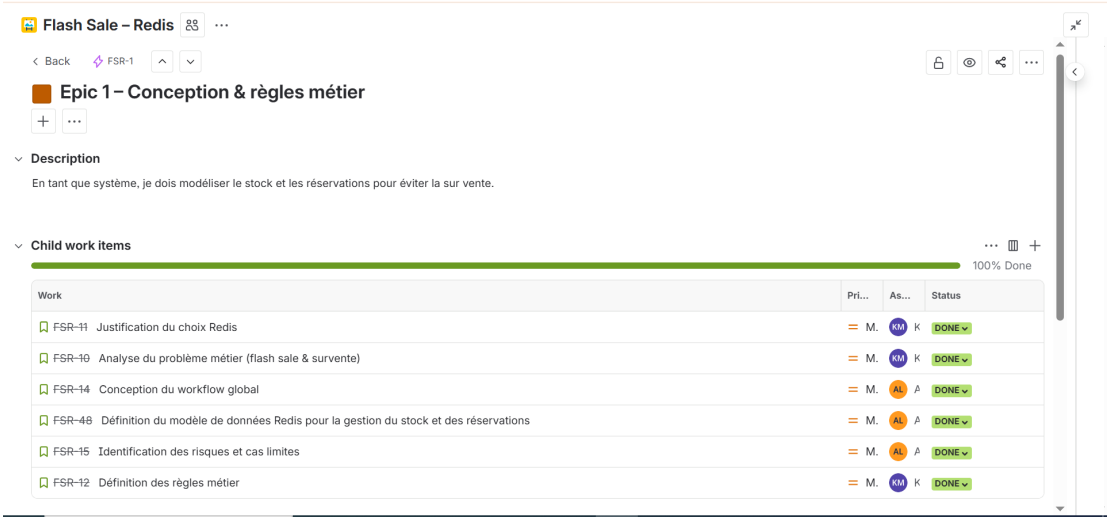


FIGURE 2 – Organisation du Sprint 1 dans Jira

- **Sprint 2 (week 2)** : Réservation TTL + atomicité Lua + annulation + libération.

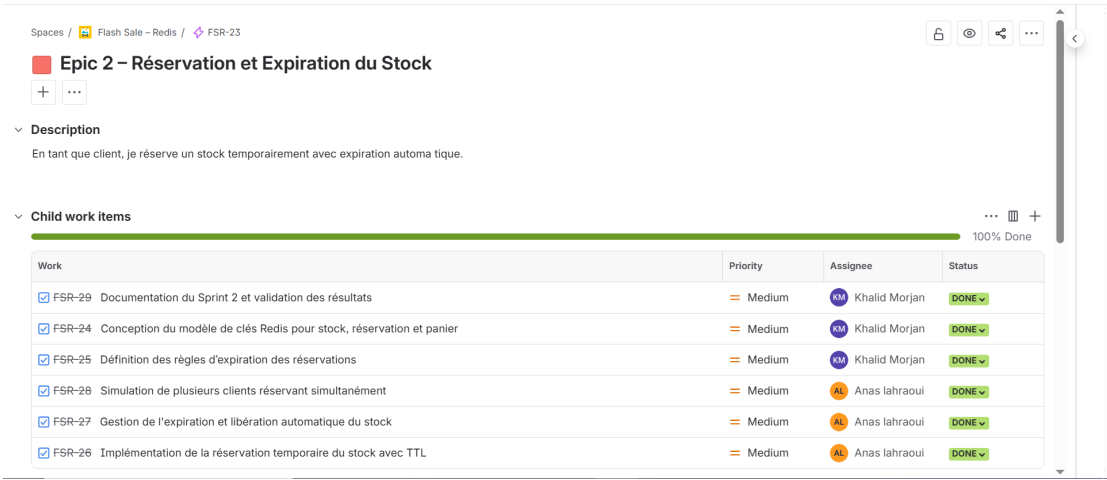


FIGURE 3 – Organisation du Sprint 2 dans Jira

— **Sprint 3 (week 3) :** Paiement idempotent + métriques + audit + scripts simulation.

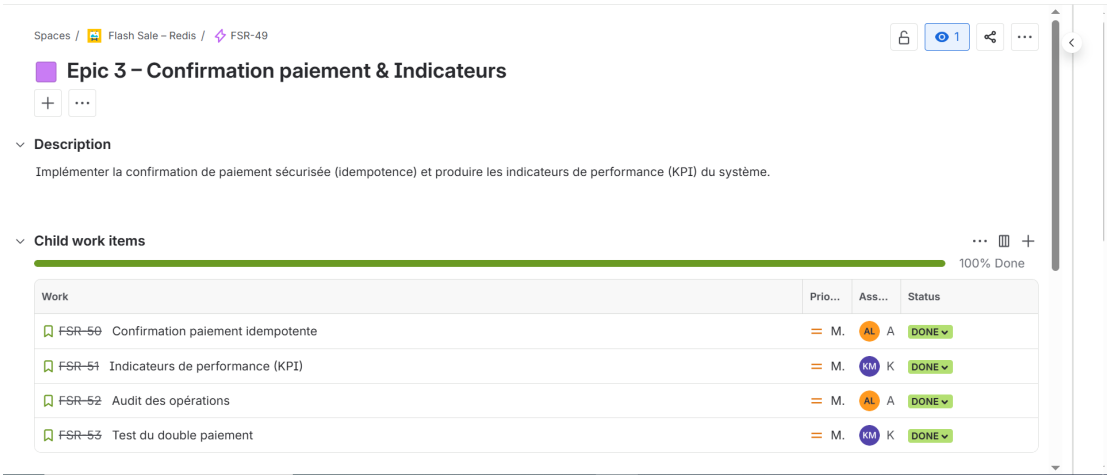


FIGURE 4 – Organisation du Sprint 3 dans Jira

— **Sprint 4 (week 4) :** Stabilisation, comparatif SAFE vs UNSAFE, résultats, rapport et annexes.

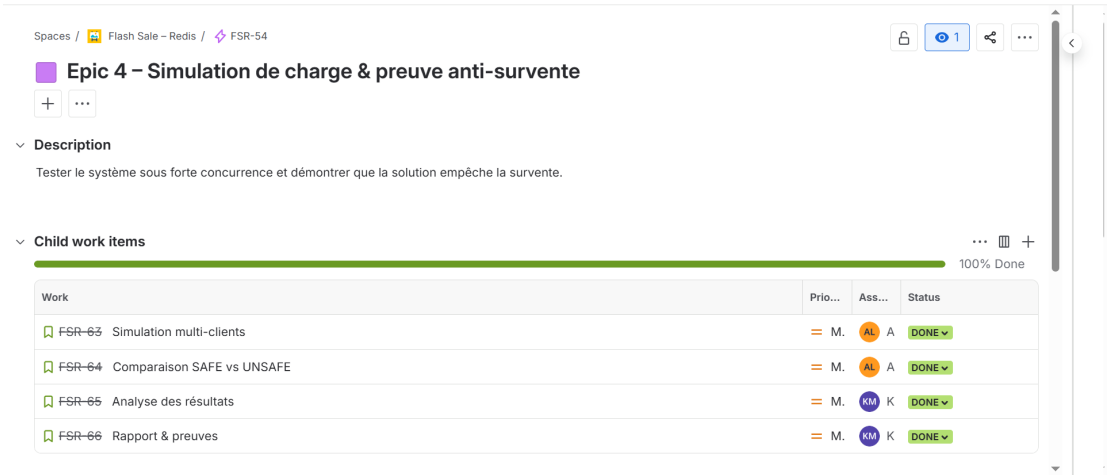


FIGURE 5 – Organisation du Sprint 4 dans Jira

- **Sprint 5** : Mettre en place tous les mécanismes permettant d’observer, mesurer et prouver le bon fonctionnement du système de vente flash.

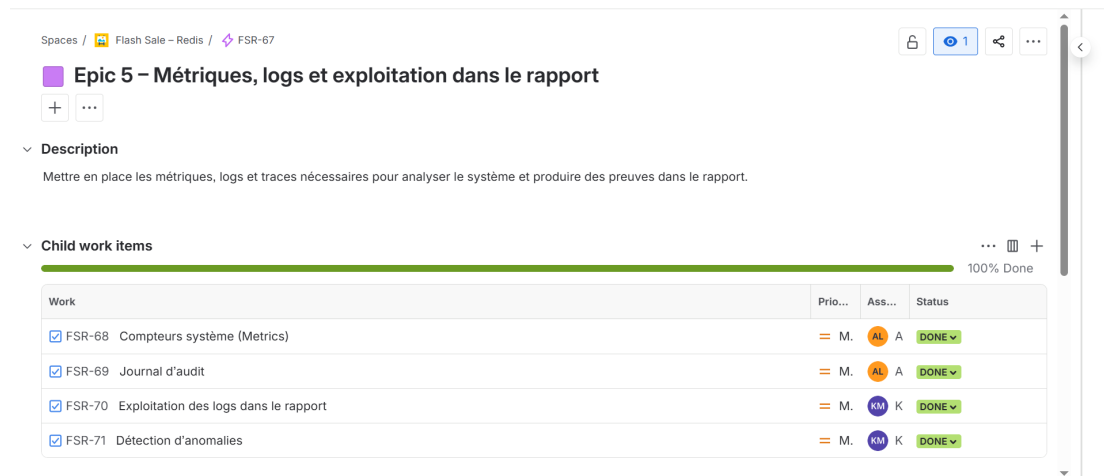


FIGURE 6 – Organisation du Sprint 5 dans Jira

4 Architecture de la solution

4.1 Vue en couches

Le projet suit une architecture simple et professionnelle en couches :

- **config.py** : centralise la configuration (host/port/db, ACL, TTL, etc.).
- **redis_client.py** : couche d’accès Redis (clés, scripts Lua, opérations atomiques).
- **app.py** : interface CLI/service qui orchestre les cas d’usage (init, reserve, confirm, metrics).

4.2 Schéma global

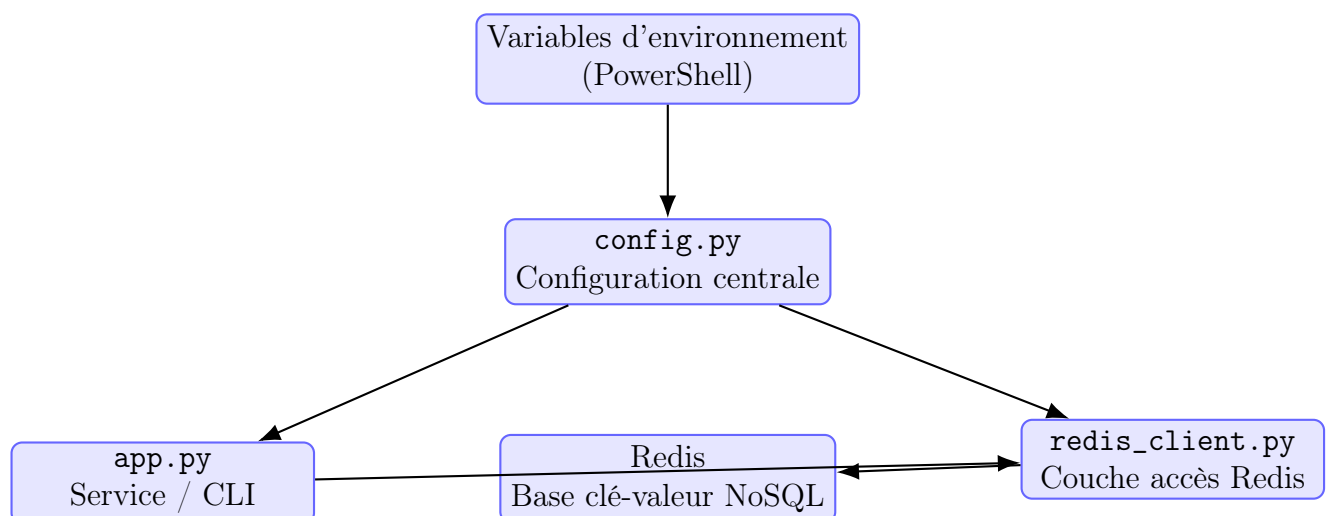


FIGURE 7 – Architecture logique du système Flash Sale basé sur Redis

4.3 Choix techniques Redis

- **TTL** pour expirer les réservations automatiquement.
- **Lua** pour garantir l'atomicité (check+update) sous concurrence.
- Structures **String/Set/Hash/List** pour stock, indexation, métriques et audit.

5 Modèle de données Redis (clés & structures)

5.1 Principe de gestion du stock réservé

On utilise le modèle :

$$available = stock_total - stock_reserved$$

où `stock_reserved` représente les unités bloquées par des paniers en cours (réservations).

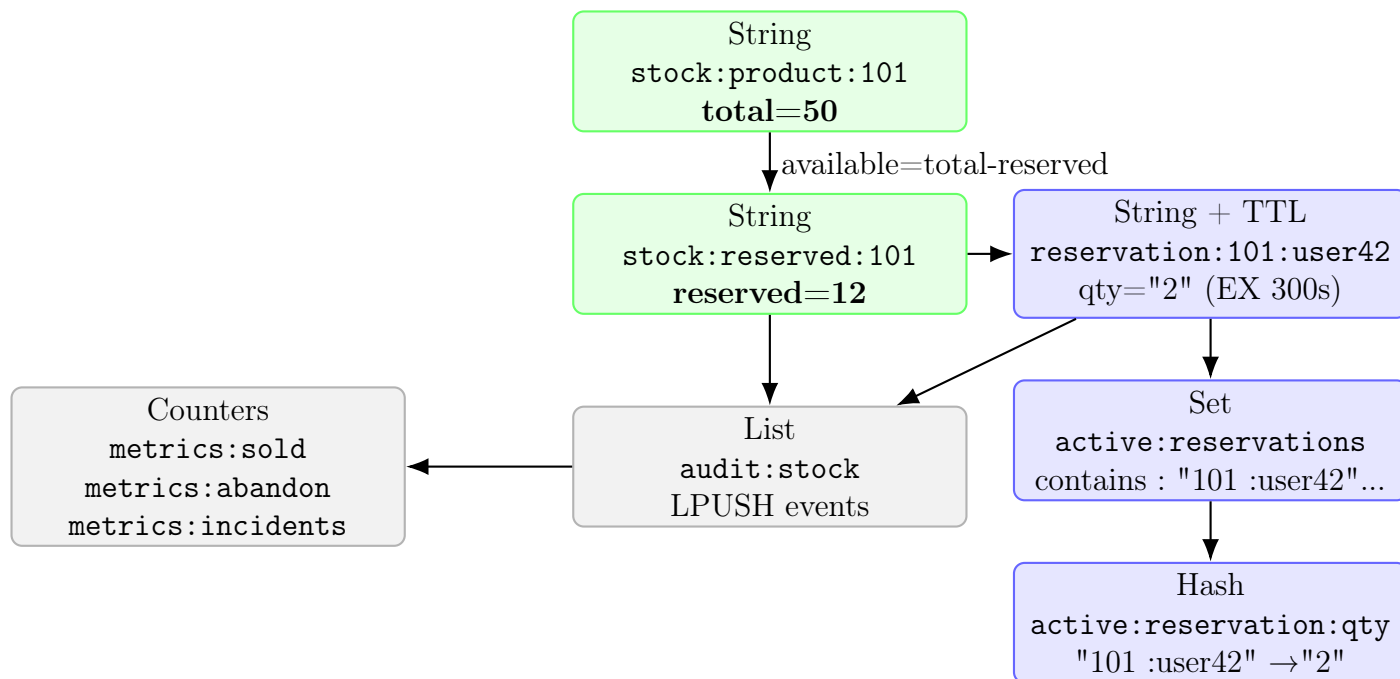


FIGURE 8 – Modèle de clés Redis : stock, réservation TTL, index, métriques et audit

5.2 Liste des clés

Clé	Type	Rôle
<code>stock:product:<id></code>	String	Stock total (capacité) du produit
<code>stock:reserved:<id></code>	String	Compteur du stock réservé
<code>reservation:<id>:<user></code>	String + TTL	Quantité réservée par user (expire)
<code>active:reservations</code>	Set	Index des réservations actives (<code>id:user</code>)
<code>active:reservation:qty</code>	Hash	Quantité associée à chaque <code>id:user</code>

<code>payment:processed:<order></code>	String	+	Idempotence contre double paiement
	TTL		
<code>metrics:sold</code>	String	counter	Nombre de paiements confirmés
<code>metrics:abandon</code>	String	counter	Nombre d'abandons (expiration)
<code>metrics:incidents</code>	String	counter	Anomalies détectées
<code>audit:stock</code>	List		Journal (audit) des opérations

5.3 Justification (structures)

- **String** : compteurs simples (INCR/DECR) et valeurs (stock, metrics).
- **Set** : indexation des réservations actives (liste unique, scan).
- **Hash** : associer rapidement la quantité à un identifiant de réservation.
- **List** : audit chronologique (LPUSH) pour traçabilité et preuves.

6 Implémentation des cas d'usage

6.1 Initialisation du stock

Objectif : initialiser `stock:product:<id>` et `stock:reserved:<id>=0`.

6.2 Réservation atomique (Lua)

Objectif : empêcher la survente. Le script Lua effectue en une seule opération :

- vérification stock disponible (`available = total - reserved`);
- création de `reservation:*` avec TTL;
- incrément de `stock:reserved:*`;
- indexation dans `active:*`;
- audit (`audit:stock`).

6.3 Annulation (cancel)

Objectif : libérer immédiatement une réservation avant expiration, et nettoyer les index.

6.4 Expiration et libération automatique (sweeper)

Objectif : si `reservation:*` expire, on relâche la quantité correspondante en s'appuyant sur :

- `active:reservations` (Set) pour lister,
- `active:reservation:qty` (Hash) pour connaître la quantité,
- mise à jour de `stock:reserved:*` + métriques + audit.

6.5 Paiement confirmé (idempotence)

Objectif : empêcher le double paiement. On crée `payment:processed:<order>` avec `SET NX EX` :

- si la clé existe déjà, on refuse (déjà traité) ;
- sinon, on finalise : décrémentation du réservé + nettoyage index + `metrics:sold++`.

7 Gestion des cas limites

- **Survente (race condition)** : évitée via Lua (atomicité check+update).
- **Double paiement** : évité via idempotence (`payment:processed:<order>`).
- **Requêtes répétées / retries** : idempotence et contrôles (déjà réservé / déjà traité).
- **Abandon tardif** : TTL + sweeper garantissent la libération même si le client disparaît.
- **Données manquantes** : compteur `metrics:incidents` et trace `audit:stock`.

8 Installation et exécution (Windows + Docker)

8.1 Prérequis

- Docker Desktop
- Python 3.11+
- Bibliothèque Python Redis : `pip install -r requirements.txt`

8.2 Démarrage Redis (Docker)

Exemple :

```
docker run -d --name redis-flash -p 6379:6379 redis
:7
```

8.3 Variables d'environnement (PowerShell, ACL Redis)

```
$env:REDIS_USERNAME="khalid"
$env:REDIS_PASSWORD="5002"
```

8.4 Commande de santé

```
python app.py ping
```

9 Tests, simulation de charge et preuves

9.1 Simulation multi-clients (simulate_clients.py)

But : simuler N clients en parallèle (reserve/confirm/cancel/abandon) et produire KPI.
Commande :

```
python 2_Simulation/simulate_clients.py --product
101 --stock 50 --clients 200 --ttl 8 \
--confirm-rate 0.6 --cancel-rate 0.1 --abandon-rate
0.3 --sweeper-interval 2
```

9.2 Comparatif SAFE vs UNSAFE (scenario_concurrency.py)

But : prouver expérimentalement la survente si on n'utilise pas l'atomicité.

```
python 2_Simulation/scenario_concurrency.py --
product 101 --stock 20 --clients 200 --ttl 8 --
unsafe-delay-ms 5
```

9.3 Interprétation des résultats

- **SAFE (Lua)** : reserved ne dépasse jamais total $\Rightarrow$ PASS.
- **UNSAFE (naïf)** : reserved > total ou available < 0 $\Rightarrow$ FAIL (survente).

9.4 Logs et preuves

Les commandes et sorties principales sont archivées dans 2_Simulation/results_logs.txt.

10 Planification (Gantt)

Remarque : un Gantt synthétise les tâches par sprint.

SPRINT	WEEK 1	WEEK 2	WEEK 3	WEEK 4
SPRINT 1	S1 : Analyse & modèle Redis			
SPRINT 2		S2 : TTL + <u>Lua</u> + Cancel		
SPRINT 3			S3 : Paiement + KPI	
SPRINT 4				S4 : Simulation + Rapport

FIGURE 9 – Planning projet (Gantt) — Sprints W1 à W4

11 Conclusion

Ce projet démontre qu'un système de vente flash peut être sécurisé contre la survente en combinant :

- **Redis** (NoSQL clé-valeur) pour la performance et la simplicité ;
- **atomicité Lua** pour éliminer les conditions de course ;
- **TTL** pour gérer l'abandon panier ;
- **idempotence** pour bloquer le double paiement ;
- **métriques + audit** pour analyser et justifier les résultats.

Les simulations concurrentes fournissent une preuve expérimentale : la méthode atomique (SAFE) préserve la cohérence, tandis qu'une approche naïve (UNSAFE) peut provoquer la survente.

A Annexes

A.1 Arborescence du projet

```
FlashSale/
  app.py
  redis_client.py
  config.py
  requirements.txt
  2_Simulation/
    simulate_clients.py
    scenario_concurrency.py
    results_logs.txt
  3_Report/
```

FlashSale_Project_Report.tex
FlashSale_Project_Report.pdf

A.2 Commandes de démo

```
# Variables ACL (PowerShell)
$env:REDIS_USERNAME="khalid"
$env:REDIS_PASSWORD="5002"

# Sante
python app.py ping

# Demo stock + reservation TTL + sweeper
python app.py init-stock --product 101 --total 5
python app.py reserve --product 101 --user user42
    --qty 2 --ttl 10
python app.py status --product 101
python app.py sweep-once
python app.py metrics
python app.py audit --n 10

# Preuve concurrence SAFE vs UNSAFE
python 2_Simulation/scenario_concurrency.py --
    product 101 --stock 20 --clients 200 --ttl 8 --
    unsafe-delay-ms 5
```

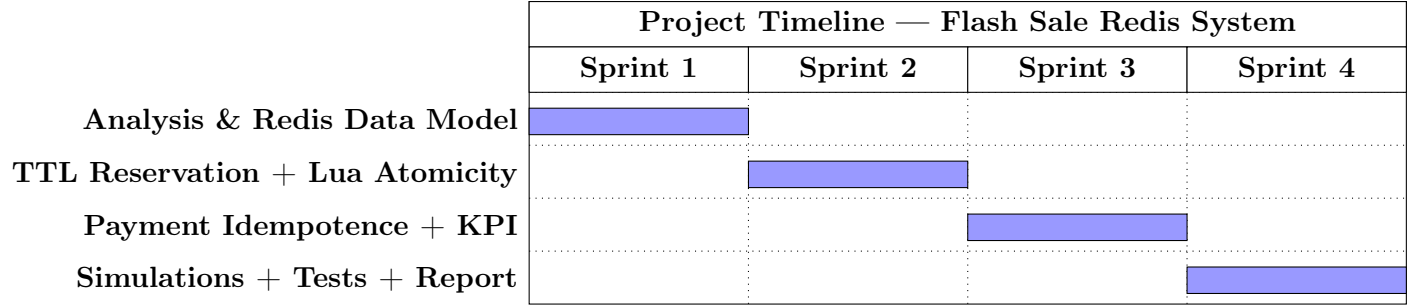


FIGURE 10 – Agile Sprint Planning of the Flash Sale System